

Hamiltonian* Path — Theoretical and Experimental Analysis

Ali Ayhan Gunder (2021400219)
Yigit Memceroktay (2020402006)

November 16, 2025

Experimental Analysis

In this section, we compare the running times of the three implemented algorithms (Naive, Optimized, and Bonus) with the theoretical time complexities that we analyzed earlier.

n	Naive (s)	Optimized (s)	Bonus (s)
4	0.000517	0.000027	0.000021
5	0.003216	0.000037	0.000050
6	0.039918	0.000033	0.000020
7	0.998846	0.000051	0.000078
8	32.163664	0.000136	0.000057
9	—	0.000187	0.000474
10	—	0.000695	0.000727
11	—	0.003766	0.001961
12	—	0.033227	0.009237
13	—	1.790858	0.009761

Table 1: Execution times for different algorithms.

Part 1: Naive Algorithm Analysis

Observed Behaviour

The running time of the Naive algorithm increases extremely fast as n grows:

- From $n = 4$ to $n = 5$: time goes from about 0.0005 s to 0.0032 s (around $6\times$).
- From $n = 5$ to $n = 6$: it jumps to about 0.04 s (around $12\times$).
- From $n = 6$ to $n = 7$: it jumps to about 1 s (around $25\times$).
- From $n = 7$ to $n = 8$: it explodes to about 32 s (more than $30\times$ again).

By $n = 8$, the algorithm already needs on the order of half a minute per instance, and running it for $n = 9$ and above would take too long to be practical. This is a clear sign of combinatorial explosion: each increase of 1 in n multiplies the running time by a large and rapidly growing factor.

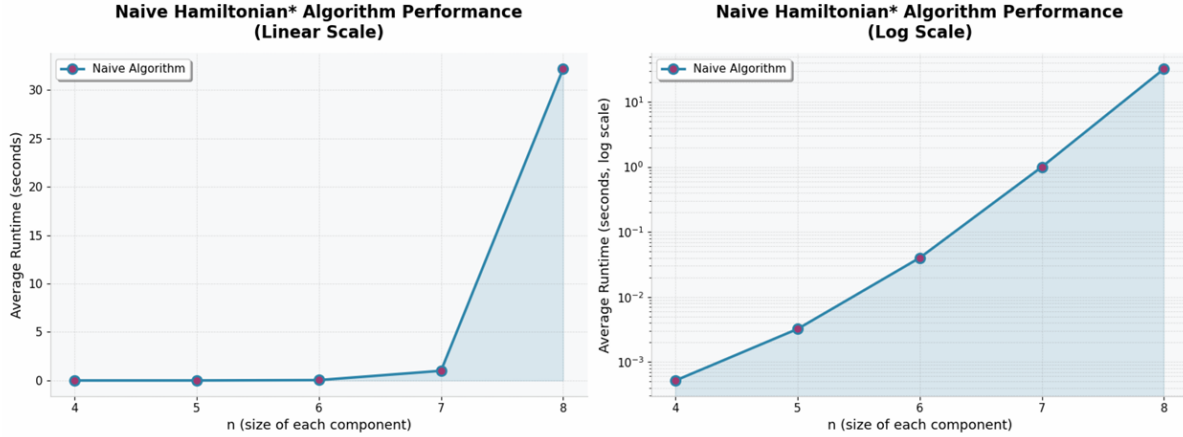


Figure 1: Naive Hamiltonian Algorithm Performance

Comparison with Theory

Earlier, we showed that the Naive algorithm has:

- Best case: $\Theta(n^2)$
- Worst and average case: $\Theta\left(\binom{3n-2}{n-2} (n-1)!\right)$

This means that in the average and worst situations, the running time grows faster than a pure factorial: there is a factorial term $(n-1)!$ multiplied by a large binomial coefficient in n . This kind of bound predicts that even small increases in n cause enormous increases in time.

The measurements match this picture very well. The growth is much faster than any polynomial, and the algorithm becomes unusable for even moderately large n . The experimental data strongly supports the theoretical analysis for the Naive algorithm.

Basic Operation

We define the basic operation as checking whether an edge exists between two consecutive vertices in a permutation, for example:

```
if H[perm[i]][perm[i+1]] == 0.
```

This is a $\Theta(1)$ comparison inside the innermost loop.

Time Complexity Analysis

Worst Case. Steps in the algorithm:

1. Generate all $\binom{3n-2}{n-2}$ subsets of size n . Only subsets containing both *start* and *end* are processed, giving

$$M = \binom{3n-2}{n-2}$$

valid subsets.

2. For each valid subset:

- Build subgraph H : $\Theta(n^2)$.
- Locate s and t : $\Theta(n)$.
- Test all permutations starting at s and ending at t : $(n-2)!$ permutations.

3. Each permutation may examine up to $n-1$ edges.

Thus the running time in the worst case is

$$T_{\text{worst}}(n) = \Theta\left(M \cdot (n^2 + n + (n-2)!(n-1))\right).$$

Since $(n-2)!(n-1) = (n-1)!$ dominates the lower-order terms, we obtain

$$T_{\text{worst}}(n) = \Theta\left(\binom{3n-2}{n-2} (n-1)!\right).$$

Best Case. The first subset tried contains a valid Hamiltonian* path and the first permutation is correct. The cost is: build H ($\Theta(n^2)$) and check one path ($\Theta(n)$). Thus:

$$T_{\text{best}}(n) = \Theta(n^2).$$

Average Case (Expected Value Method). Now we analyze the average number of permutations tested using the expected value method.

Let

$$M = \binom{3n-2}{n-2}$$

be the number of valid subsets (size n , containing *start* and *end*). For each such subset there are $(n-2)!$ permutations that start at s and end at t . Therefore, the total number of permutations that the algorithm may test is

$$N = M (n-2)! = \binom{3n-2}{n-2} (n-2)!.$$

We assume the following probabilistic model (as in the lecture slides for linear search):

- There is exactly one “correct” permutation among these N permutations (that is, one Hamiltonian* path).
- All positions of this correct permutation in the test order are equally likely.

Let the random variable K be the position of the correct permutation in the sequence of tests. Then K is uniformly distributed on $\{1, 2, \dots, N\}$, and the algorithm tests exactly K permutations before it terminates.

The expected value of K is

$$\mathbb{E}[K] = \frac{1}{N} \sum_{k=1}^N k = \frac{1}{N} \cdot \frac{N(N+1)}{2} = \frac{N+1}{2} \in \Theta(N).$$

Each permutation test checks at most $n-1$ edges, so the number of basic operations due to permutation testing is at most

$$(n-1)K.$$

Thus, by linearity of expectation,

$$\mathbb{E}[\text{basic operations from permutations}] = (n-1) \mathbb{E}[K] \in \Theta((n-1)N).$$

Substituting $N = \binom{3n-2}{n-2} (n-2)!$,

$$\mathbb{E}[\text{basic operations}] \in \Theta\left((n-1) \binom{3n-2}{n-2} (n-2)!\right) = \Theta\left(\binom{3n-2}{n-2} (n-1)!\right),$$

since $(n-1)(n-2)! = (n-1)!$.

The cost of building subgraphs H for each subset is

$$M \cdot \Theta(n^2) = \Theta\left(\binom{3n-2}{n-2} n^2\right),$$

which is asymptotically smaller than the permutation cost above. Therefore the total average running time is

$$T_{\text{avg}}(n) = \Theta\left(\binom{3n-2}{n-2} (n-1)!\right).$$

Summary for the Naive Algorithm.

- Best case: $\Theta(n^2)$
- Worst case: $\Theta\left(\binom{3n-2}{n-2} (n-1)!\right)$
- Average case: $\Theta\left(\binom{3n-2}{n-2} (n-1)!\right)$

Part 2: Optimized Algorithm Analysis

Observed Behaviour

The Optimized algorithm behaves much better:

- For n from 4 to 9, the times are extremely small and almost flat, staying between about 0.000027 s and 0.000187 s.
- Starting around $n = 10$, the running time begins to grow more clearly:
 - $n = 10$: 0.000695 s
 - $n = 11$: 0.003766 s (about $5.4\times$ larger than for $n = 10$)
 - $n = 12$: 0.033227 s (about $8.8\times$ larger than for $n = 11$)
 - $n = 13$: 1.790858 s (about $54\times$ larger than for $n = 12$)

For small n , the algorithm is basically instant; then the exponential behaviour starts to show up around $n = 11$ – 13 .

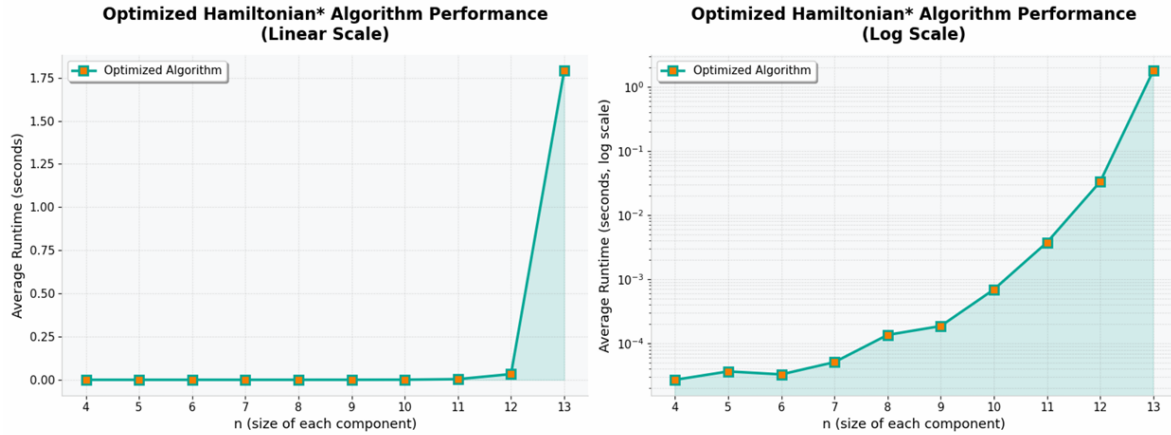


Figure 2: Optimized Hamiltonian Algorithm Performance

Comparison with Theory

For the Optimized algorithm, the theoretical analysis gave:

- Best case: $\Theta(n^2)$
- Average and worst case: $\Theta((n-1)!)$

The key improvement compared to the Naive algorithm is that we eliminated the factor that comes from enumerating all subsets of size n . Instead of working over all subsets of the $3n$ vertices, the Optimized algorithm immediately finds the relevant component of size n and only explores that.

The data fits this story well:

- For small n (4–9), the factorial part has not “kicked in” yet, so the cost is dominated by relatively small constant work (BFS/DFS, building the component subgraph, etc.), and the times look almost constant.
- For larger n (10–13), the runtime starts to grow very quickly, as expected from a factorial-type bound.
- Compared to the Naive algorithm, the Optimized one pushes the practical limit to much larger n . Naive explodes at $n = 8$, while Optimized only becomes heavy around $n = 13$.

Overall, the empirical curve is consistent with the $\Theta((n-1)!)$ average and worst-case complexity that we proved for the Optimized algorithm.

Key Optimization

The graph has three disconnected components of size n . A Hamiltonian* path must:

- remain inside a single component, and
- contain exactly n vertices (the component size).

Thus *start* and *end* must be in the same component.

Algorithm Steps

1. Find the connected component of *start* using BFS/DFS on the adjacency matrix: $\Theta(n^2)$.
2. Check whether *end* is in the same component: $\Theta(1)$.
3. Restrict the search to this component of size n and build the induced subgraph H : $\Theta(n^2)$.

Time Complexity

Worst Case. Component detection takes $\Theta(n^2)$, building the induced subgraph costs $\Theta(n^2)$, and testing permutations takes $(n-2)!$ trials, each checking up to $n-1$ edges:

$$T_{\text{worst}}^{\text{opt}}(n) = \Theta(n^2) + \Theta(n^2) + \Theta((n-2)!(n-1)) = \Theta((n-1)!).$$

This improves on the naive $\Theta\left(\binom{3n-2}{n-2}(n-1)!\right)$ by removing the exponential subset generation factor.

Best Case. Same as the Naive case (but restricted to a single component):

$$T_{\text{best}}^{\text{opt}}(n) = \Theta(n^2).$$

Average Case. The average number of permutations examined inside the component is still factorial in n , so:

$$T_{\text{avg}}^{\text{opt}}(n) = \Theta((n-1)!).$$

Comparison Summary

Algorithm	Best Case	Worst Case	Average Case
Naive	$\Theta(n^2)$	$\Theta\left(\binom{3n-2}{n-2}(n-1)!\right)$	$\Theta\left(\binom{3n-2}{n-2}(n-1)!\right)$
Optimized	$\Theta(n^2)$	$\Theta((n-1)!)$	$\Theta((n-1)!)$

Bonus Part 3: Held–Karp Algorithm Analysis

After doing some research, we found an algorithm named the Held–Karp algorithm that performs better than our Naive and Optimized algorithms. Therefore, for Part 3, we decided to use the Held–Karp algorithm as our bonus algorithm. Below is a detailed analysis.

In the experiments, we chose our bonus algorithm to be a Held–Karp-style dynamic programming algorithm that runs on the component containing *start* and *end* and uses a subset-DP table of size $\Theta(n 2^n)$.

Observed Behaviour

The measured running times for this algorithm are:

- $n = 4$: 0.000021 s
- $n = 5$: 0.000050 s

- $n = 6$: 0.000020 s
- $n = 7$: 0.000078 s
- $n = 8$: 0.000057 s
- $n = 9$: 0.000474 s
- $n = 10$: 0.000727 s
- $n = 11$: 0.001961 s
- $n = 12$: 0.009237 s
- $n = 13$: 0.009761 s

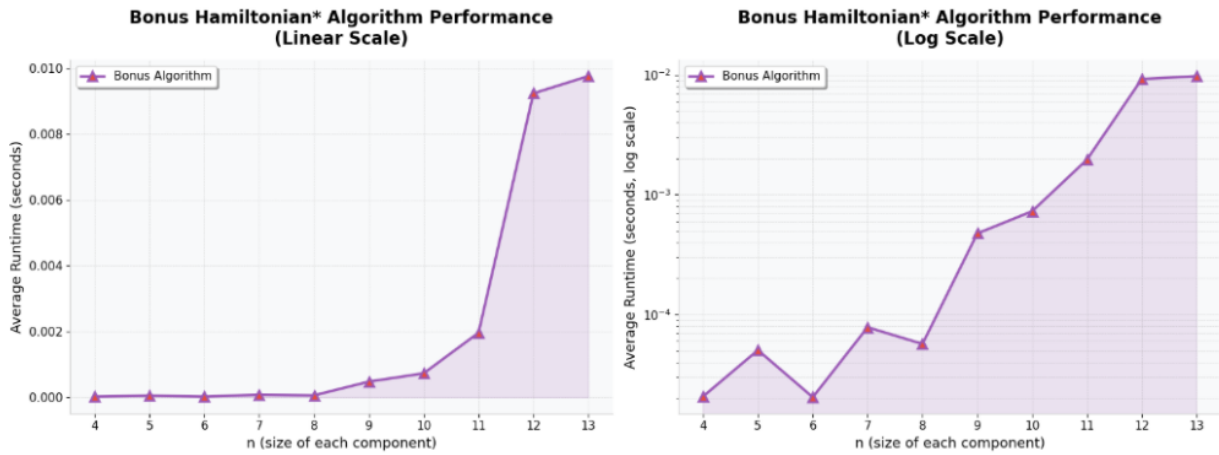


Figure 3: Bonus (Held–Karp) Hamiltonian Algorithm Performance

All of these values are extremely small, and there is no visible explosion of the running time up to $n = 13$. The times also fluctuate slightly (for example, the running time at $n = 13$ is a bit smaller than at $n = 12$), which is reasonable at this scale: when the absolute times are so small, timer resolution, OS scheduling, and randomness in the graph can easily cause small non-monotonic variations.

In practice, up to $n = 13$, the Held–Karp algorithm behaves almost like a constant-time algorithm compared to the other two: its running time is tiny and remains well below a millisecond.

Comparison with Theory

From the theoretical analysis of the Held–Karp algorithm on a single component of size n , we obtained:

- Number of DP states: $\Theta(n 2^n)$
- Work per state (transitions): $\Theta(n)$
- Overall time (best, average, and worst case): $\Theta(n^2 2^n)$

Unlike the Naive and Optimized algorithms, the Held–Karp DP does not stop early after finding one Hamiltonian* path. It systematically fills almost the entire DP table: it iterates over all subsets S and all endpoints v , and for each such state, it considers possible transitions. Therefore, its running time depends mostly on n and not very much on the actual arrangement of edges in the graph.

Held–Karp has exponential time complexity $\Theta(n^2 2^n)$, which is significantly better asymptotically than the factorial complexity $\Theta((n-1)!)$ of the Naive and Optimized algorithms.

Outline of the Algorithm

After finding the connected component of *start* and verifying that *end* lies in the same component of size n , the Held–Karp-style algorithm compresses that component to vertices $\{0, 1, \dots, n-1\}$ and builds an $n \times n$ adjacency matrix adj . It then runs a subset DP of the form

$$\text{dp}[S][v] = \begin{cases} \text{True,} & \text{if there is a simple path starting at } s \text{ and ending at } v \\ & \text{that visits exactly the vertices in } S, \\ \text{False,} & \text{otherwise,} \end{cases}$$

where $S \subseteq \{0, \dots, n-1\}$ is represented as a bitmask and s is the compressed index of *start*. The transition is

$$\text{dp}[S \cup \{w\}][w] \leftarrow \text{True} \quad \text{if} \quad \text{dp}[S][v] \text{ is True and } \text{adj}[v][w] = 1.$$

Finally, the algorithm checks $\text{dp}[\text{FULL_MASK}][t]$ for the compressed index t of *end*.

Time Complexity

The DP is executed on a single component of size n (after the initial component detection). We analyze the running time in terms of this n .

Number of states. There are 2^n possible subsets $S \subseteq \{0, \dots, n-1\}$, and for each subset there are n possible endpoints v . Thus the table dp has $\Theta(n 2^n)$ entries.

Transition cost. For each state (S, v) with $\text{dp}[S][v] = \text{True}$, the algorithm tries to extend the current path by adding any vertex $w \notin S$ with an edge $v \rightarrow w$, i.e. it considers up to n candidates w and performs an adjacency check $\text{adj}[v][w]$ for each.

In the worst case (and hence in general asymptotic analysis) we may assume that for each subset S and endpoint v the corresponding state is reachable, yielding the following upper bound on the total number of basic operations:

$$\Theta(2^n \cdot n \cdot n) = \Theta(n^2 2^n).$$

The initial component detection via BFS on the full $3n \times 3n$ adjacency matrix costs $\Theta(n^2)$, which is asymptotically dominated by $\Theta(n^2 2^n)$ for large n .

Therefore the overall running time of the Held–Karp algorithm on a component of size n is

$$T_{\text{HK}}(n) = \Theta(n^2 2^n).$$

Average Case. Unlike the Naive and Optimized algorithms, the Held–Karp DP does not stop early after finding a single Hamiltonian* path; it systematically fills (almost) the entire DP table for all subsets S and endpoints v up to size n . The set of states and the transition loops depend only on n , not on the specific edge configuration of the component.

Let $T(n, G)$ denote the running time of the algorithm on a component G of size n . From the analysis above, there exists a constant $c_2 > 0$ such that for every such graph G ,

$$T(n, G) \leq c_2 n^2 2^n.$$

If we consider the random graph model of the project, a large fraction of DP states are typically reachable, so the expected running time is

$$\mathbb{E}[T(n, G)] \in \Theta(n^2 2^n).$$

Thus, for this model, the Held–Karp algorithm has

$$T_{\text{avg}}^{\text{HK}}(n) = \Theta(n^2 2^n).$$

1 Combined Graphs

Below we show the results of the three algorithms combined in one graph, given in both linear scale and logarithmic scale.

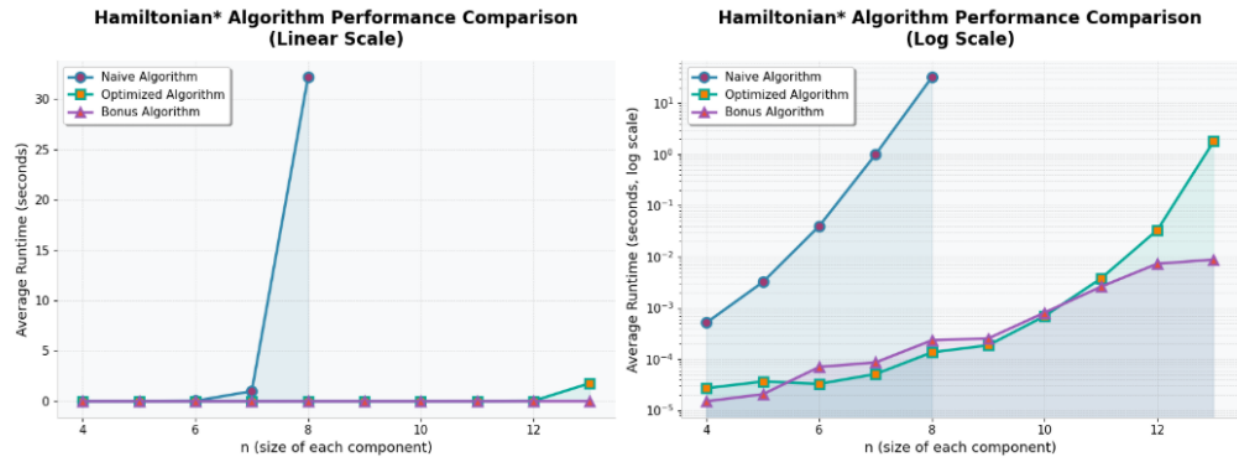


Figure 4: Algorithms Performance Comparison (linear and log scales)

We also include a bar graph that shows how much faster the Optimized and Bonus algorithms are compared to the Naive approach (in log scale). As we can see, both the Optimized and Bonus algorithms provide substantial speedup as n gets larger.

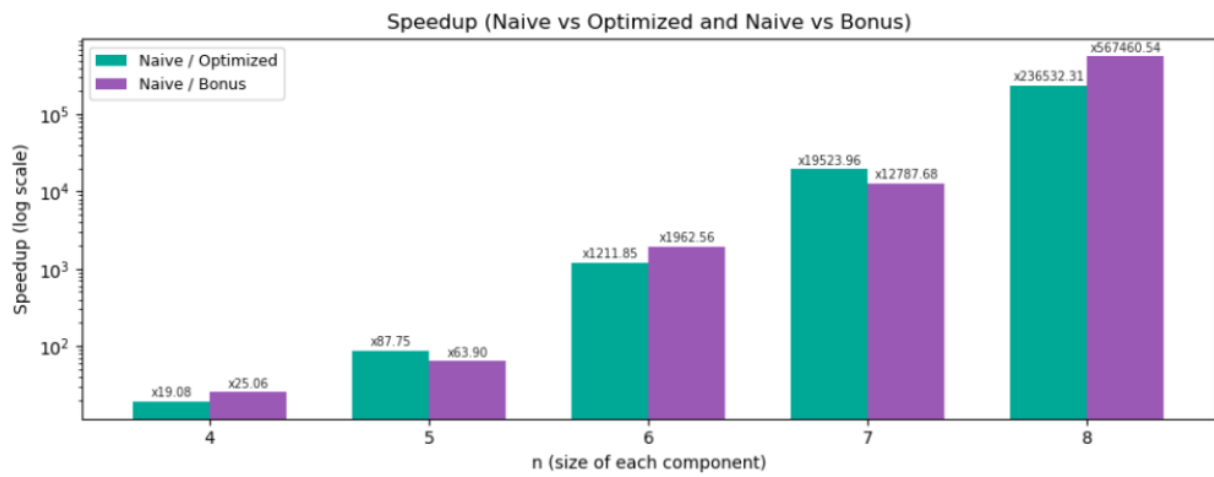


Figure 5: Algorithms Speedup Comparison (log scale)